

USENIX

THE ADVANCED COMPUTING
SYSTEMS ASSOCIATION

KINTSUGI: Secure Hotpatching for Code-Shadowing Real-Time Embedded Systems

Philipp Mackensen, *Ruhr University Bochum;* Christian Niesler, *University of Duisburg-Essen;* Roberto Blanco, *Eindhoven University of Technology/MPI-SP;* Lucas Davi, *University of Duisburg-Essen;* Veelasha Moonsamy, *Ruhr University Bochum*

https://www.usenix.org/conference/usenixsecurity25/presentation/mackensen

This paper is included in the Proceedings of the 34th USENIX Security Symposium.

August 13–15, 2025 • Seattle, WA, USA

978-1-939133-52-6

Open access to the Proceedings of the 34th USENIX Security Symposium is sponsored by USENIX.

ARTIFACT EVALUATED
usenix ASSOCIATION
AVAILABLE

KINTSUGI: Secure Hotpatching for Code-Shadowing Real-Time Embedded Systems

Philipp Mackensen[*], *Christian Niesler*[†], *Roberto Blanco*[‡], *Lucas Davi*[†], *Veelasha Moonsamy*[*]
[*]*Ruhr University Bochum,* [†]*University of Duisburg-Essen,* [‡]*Eindhoven University of Technology/MPI-SP*

Abstract

Mission-critical embedded devices deal with strict real-time constraints, and thus make traditional updates or reboots unsuitable. While runtime fixes (i.e., hotpatching) reduce downtime, they pose challenges for resource management and real-time performance. Previous work has focused mainly on hotpatching devices executing their firmware from flash, neglecting those that use code-shadowing to execute firmware from RAM. These approaches neglect secure end-to-end hotpatch deployment during runtime, putting vulnerable devices at risk.

We introduce KINTSUGI[1], the first secure hotpatching framework for real-time embedded devices that uses code-shadowing. By leveraging the context switch of real-time operating systems, we achieve atomic application of hotpatches while enforcing strict memory policies to protect KINTSUGI's resources with minimal overhead. KINTSUGI is designed to prevent tampering attacks on *both* the framework and deployed hotpatches. Evaluated on the *NRF52840-DK* with an ARM Cortex-M4 MCU running at 64 MHz, a processor deployed in millions of devices, our results demonstrate KINTSUGI's performance advantage with overheads as low as 38 cycles (0.59μs) during normal operation, peaking at 216 cycles (3.38μs). We show KINTSUGI's effectiveness addressing real-world vulnerabilities in popular real-time operating systems like FreeRTOS and Zephyr, and libraries such as mbedTLS and picoTCP. Our approach introduces negligible overhead, making it ideal for real-time applications, as illustrated by our case study.

1 Introduction

Real-time devices must operate uninterrupted and without downtime, while maintaining high accuracy and predictability. To meets these requirements, they typically run a real-time operating system (RTOS) like Zephyr [15] and FreeRTOS [7]. As the internet of things (IoT) continues to grow [69], these devices become increasingly interconnected, and consequently more susceptible to cyberattacks. The number of attacks targeting IoT devices rises every year [34, 58], and vulnerabilities like AMNESIA:33 [16], Ripple20 [17], and URGENT/11 [66], continue to pose significant threats. In particular, rapid response through security updates is critical to ensuring safety and security of real-time embedded devices. Traditional update mechanisms often require system reboots and downtime, making them impractical for real-time embedded devices. *Hotpatching*, a method to apply small, volatile patches to a running system on-the-fly, has emerged as a promising technology. However, adapting hotpatching to real-time embedded devices presents significant challenges, notably maintaining concurrent execution of the RTOS without disrupting other tasks and introducing unpredictable delays.

Several approaches [40, 53, 62] investigated hotpatching mechanisms for real-time embedded devices, adhering to their strict timing constraints. Although these works address various technical challenges in hotpatching, they share two key limitations. First, *code-shadowing*, a widely used technique used in millions of IoT devices [23, 64, 68] and real-time programmable units (RPUs) [24], which executes firmware by loading it from non-volatile flash into random-access memory (RAM) has been ignored. Second, while there has been research on developing secure update mechanisms, including patch acquisition protocols [31, 35, 54], update mechanisms [18, 22, 57, 67] and patch deployment [43, 49], the security of hotpatch deployment and the hotpatching mechanism itself are frequently neglected. Previous work adopts overly optimistic security assumptions about their frameworks and execution environments. For instance, existing approaches [40, 53, 59, 62] overlook memory protection, allowing attackers to alter hotpatches to gain code execution privileges or reintroduce vulnerabilities.

In this paper, we first introduce and analyze a naïve approach that aims to mitigate vulnerabilities on a real-time device with code-shadowing. In doing so, we seek to identify the main challenges of managing the real-time constraints of the underlying RTOS while applying hotpatches without

[1] *Kintsugi* refers to the Japanese art of putting broken pottery pieces back together with gold, embracing their history rather than hiding it.

disrupting the device's operation. We also considered the necessary security requirements by splitting our analysis into three stages: *before*, *during*, and *after* the application of hotpatches. For each of these stages, we identify ways in which an attacker could tamper with the hotpatching process itself or the hotpatch stored in memory. From these insights, we define security goals that our proposed hotpatching framework guarantees, while balancing security and performance.

Our framework, KINTSUGI, separates the management and application of hotpatches into two distinct components. First is the *Manager*, which processes incoming hotpatches. Second is the *Applicator*, which achieves the atomic application of hotpatches to the firmware. KINTSUGI secures the end-to-end hotpatching process by enforcing strict memory policies on isolated memory regions, using the memory protection unit (MPU) found in many ARM Cortex processors [19, 20]. By design, only the *Manager* and *Applicator* can access these memory regions. This enforcement is achieved by integrating a Guard component that grants or revokes access to these memory regions depending on the current execution state.

We implement a prototype of KINTSUGI and integrate it into two popular RTOSes, FreeRTOS [7] and Zephyr [15] on the `nRF52840-DK` board. The board contains an ARM Cortex-M4 micro-controller unit (MCU) running at 64MHz and is part of the Cortex-M family that has been shipped several billion times [2, 65]. We evaluate KINTSUGI's performance by micro-benchmarking each component, observing overheads between 38 cycles (0.59μs) to 216 cycles (3.38μs), which have negligible impact on context switch performance. Additionally, we implement hotpatches for 10 real-world common vulnerability exposures (CVEs), demonstrating KINTSUGI's ability to address real vulnerabilities. Further, we evaluate the security of KINTSUGI itself by performing a theoretical and practical security analysis. We demonstrate that our framework remains secure even in the presence of an attacker focusing the framework or hotpatches. We conclude our evaluation with a case study on a drone (Crazyflie 2.0 [4], widely used in robotics research [37]), showcasing KINTSUGI's integration into real-world devices by applying hotpatches, despite strict real-time constraints, with minimal impact on performance.

To summarize, we make the following contributions:

- We propose a general approach to hotpatching, and use it to derive the security goals for our framework (§ 3).
- We design (§ 4) and implement (§ 5) KINTSUGI, the first end-to-end secure hotpatching framework for real-time embedded systems with code-shadowing.
- We evaluate our prototype on two popular RTOSes (§ 6).
- We analyze the performance, security (§ 6, § 7 and § 8), and real-world applicability (§ 9) of KINTSUGI.

To foster future research, we make KINTSUGI's prototype implementation open source at `https://doi.org/10.5281/zenodo.15592036`.

2 Background

Hotpatching Real-Time Devices. Hotpatching is a technique that allows devices to apply hotpatches, i.e., minor, temporary fixes to their firmware at runtime without interrupting the device's functionality. It is commonly found in Android, Linux, or Windows and used to apply small fixes to mitigate firmware vulnerabilities. In real-time embedded devices, hotpatching a vulnerability closes its vulnerable exposure window. Because of their real-time requirements, these devices cannot reboot at will and have a fixed schedule for applying firmware updates. Therefore, if patches become available between update periods, the devices remain vulnerable until their next scheduled update. A hotpatching system can bridge the time gap between updates and apply minor fixes to reduce the potential damage an exploit could cause to a vulnerable device. A hotpatching mechanism must accommodate the timing requirements of the underlying RTOS and integrate seamlessly, behaving predictably so that developers can safely deploy it without risking potential crashes while applying a hotpatch to the firmware.

Memory Protection Unit. The MPU is a hardware unit in ARM Cortex-M and Cortex-R processors that provides access controls over memory regions. Depending on the underlying processor, the MPU supports between 8 to 16 configurable regions. Each of these regions can be individually set up to enforce policies on a specified memory range. These memory regions can also overlap, with an index determining their priority level. Say, if region 2 overlaps with region 0, then the configuration of region 2 will take place for the overlapping portion of memory. The unprivileged mode must always have the same or more restrictive access than the privileged mode. It is also possible to enforce memory execution control by setting the eXecute Never (XN)-bit. Conversely, free access to regions can be fully allowed by setting the access permission (AP)-bit. These controls enable to configure an MPU region as read/write-execute (RWX), read/write (RW), read-execute (RX), read-only (RO) or no-access (NA). Aaccess to memory regions protected by the MPU, such as a write to a region configured as read-only, will result in exceptions that can then be caught and handled by the developer.

3 Problem Statement & Goals

Ensuring the secure and timely application of hotpatches to real-time embedded devices is a critical challenge. Hotpatching frameworks must be designed to resist tampering attacks. This is because they are in a highly privileged position within the firmware, allowing them to modify it arbitrarily. At the same time, implementing these security measures should not result in an unacceptable increase in execution time. Careful design and alignment of security and real-time constraints are required to achieve this balance.

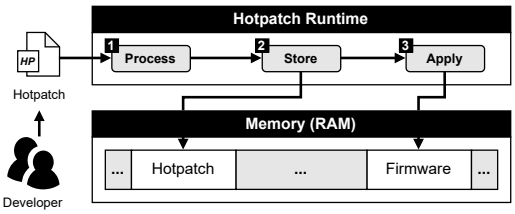


Figure 1: An example of the workflow of a naïve hotpatching approach showing the operational workflow and effects on memory.

3.1 Assumptions & Threat Model

Assumptions. We assume an embedded device equipped with a processor that supports hardware privilege separation through an MPU. The device performs code shadowing and thus runs firmware that is loaded into RAM for execution. This loading process can occur either via a bootloader or through a remote processor framework that transfers the firmware to the device's memory. We also assume that the firmware is based on a conventional RTOS that utilizes fixed-priority preemptive scheduling (FPPS), a widely used algorithm for task scheduling [5, 6, 11, 13]. The RTOS supports varying real-time constraints, such as soft, firm, and hard real-time requirements. Once the firmware is loaded into RAM, the MPU is configured to enforce memory protection policies such as the popular $W \oplus E$ policy, which ensures that writable memory regions cannot be executed at the same time. We also assume that the device is connected to a local network. Hotpatches are transmitted over a secure channel that enforces encryption and digital signatures to maintain integrity and authenticity of the hotpatches, during transmission.

Threat Model. We consider a remote attacker without physical access to the target device. The attacker identifies a vulnerability in the firmware through, e.g., reverse engineering or source code analysis. This vulnerability could be a memory corruption issue, such as a buffer overflow, or arise from a logical flaw, such as improper input validation. A task running in the RTOS executes the function that contains the vulnerability that the attacker aims to compromise.

Once the attacker has control over the unprivileged task, their goal is to exploit the hotpatching framework. They may attempt to reintroduce vulnerabilities, inject new vulnerabilities, or modify existing hotpatches to execute malicious code. However, the attacker does not have credentials to transfer maliciously crafted hotpatches directly to the device, to modify the firmware. We emphasize that our threat model does not cover validating the functionality, safety, or security of the hotpatch code, as these aspects are beyond the scope of this work. Our focus is on ensuring that hotpatches can be securely applied to the firmware even in the presence of a compromised unprivileged task.

3.2 Naïve Hotpatching

We first consider a naïve approach to a hotpatching framework to understand the real-time challenges and highlight the inherent security risks of hotpatching. The hotpatch runtime is illustrated in Figure 1, including the hotpatch workflow and its effects on memory. The runtime itself is executed as a low-priority task within the firmware to avoid interfering with critical, deadline-driven tasks. When the device receives a hotpatch, it is passed to the hotpatch runtime, which then *processes* 1 the hotpatch (i.e., adjusting it to the dynamic memory layout). This step initiates the patching process and creates a boundary within which the hotpatch transitions from an external, untrusted input to an internal, trusted resource of the framework. The framework itself assumes that the patch has been verified and transmitted over a secure channel; these are common assumptions [40, 53, 59, 62]. Then, it dynamically allocates memory to *store* 2 the hotpatch, effectively assuming ownership and ensuring control over the allocated memory space. Finally, the framework *applies* 3 the hotpatch to the vulnerable firmware by inserting a trampoline into the affected code region. The trampoline redirects the execution from the vulnerable firmware to the stored hotpatch code. This modification is typically implemented by inserting branch instructions at the targeted address. As a result, the hotpatch is applied to the firmware, mitigating the targeted vulnerability.

3.3 Real-Time Integrity

In real-time embedded devices with a FPPS RTOS, tasks can be interrupted at any given time by a higher-priority task. This dynamic scheduling presents significant challenges when hotpatches are applied at runtime. In the previous naïve approach, the hotpatching framework runs as a low-priority task, which means that the RTOS can interrupt task execution while the framework is in the process of applying a hotpatch. Such interrupts can result in partial application of hotpatches to vulnerable firmware. If another task then attempts to execute the vulnerable code, the system may experience undefined behavior, corruption of the system state, or system crashes. This illustrates how partially applied hotpatches can introduce race conditions, posing a serious threat to the system's stability.

Addressing the challenge of applying hotpatches in an RTOS without causing race conditions is crucial. To this end, the framework must ensure that hotpatches are applied atomically, guaranteeing that they are either fully applied or not applied at all. One way to achieve atomicity is to execute the patch application process within a critical section, during which interrupts are temporarily disabled. However, in the context of our naïve approach, this is infeasible. If a high-priority task requires immediate execution but cannot preempt the hotpatching task due to the critical section preventing a task switch, this can cause unpredictable delays and viola-

tions of real-time guarantees. Such delays can halt essential operations of the system and compromise the timely execution requirements of the RTOS, making traditional critical sections unsuitable for real-time systems. As a result of the real-time constraints and difficulty of applying hotpatches in a real-time preserving matter, we define the following goal for our proposed hotpatching framework.

Goal 1: Real-Time Preserving Hotpatch Application

Preserve the real-time integrity of the RTOS while applying a hotpatch to the firmware. This implies that the hotpatch must be applied atomically, without introducing unexpected delays, and ensuring that tasks meet their deadlines.

3.4 From Insecure to Secure Hotpatching

One of the biggest security threats to hotpatching frameworks is tampering, as it allows attackers to manipulate the patching process or the hotpatches themselves to their advantage. A successful tampering attack can result in, for instance, malicious code injections or changes in the execution state. Concerns arise when dealing with code-shadowing firmware due to the risk of modification of executable code in RAM. Hotpatches are typically stored in dynamically allocated memory that lasts until a reboot. Without adequate security measures, these hotpatches can be manipulated by attackers to execute arbitrary code. Since the framework, by nature, allows for firmware modifications, it is essential to prevent attackers from compromising it to alter the firmware.

Therefore, we want to identify potential security threats in order to establish suitable security goals. We accomplish this by dividing the naïve hotpatching approach into three stages: *before* patching, when the framework receives and stores the hotpatch; *during* patching, when the framework actively applies the hotpatch to the firmware; and *after* patching, when the hotpatch remains in volatile memory until system reboot. We will provide a brief explanation of each stage, outline potential attack vectors, and define the overarching security goals that our proposed framework must achieve.

Before Patching. Before a hotpatch is applied to the firmware, it must be received by the device, passed on to the hotpatching framework and stored in memory controlled by the framework. This stage is highly vulnerable to tampering attacks and the supply of malicious crafted hotpatches. Attackers may attempt to intercept the patch before it reaches the framework or modify it before it is stored. By injecting malicious code or altering the metadata of the patch, an attacker may compromise the integrity of the patch, effectively turning a security update into a new vulnerability.

Goal 2: Verification & Validation

Ensure the authenticity and integrity of hotpatches through security and validation checks, preventing interception or unauthorized modification before patching.

During Patching. The patch application process itself introduces additional risks because the framework must retrieve and process information about the patch to modify the vulnerable firmware. At this stage, an attacker could manipulate the runtime context of the hotpatching framework, changing the target location in the firmware to be patched or corrupting the data being written. This tampering could result in incomplete or malicious updates, potentially leading to undefined behavior and even giving the attacker control over critical system components. In addition, race conditions pose a significant threat due to the preemptive RTOS and add a layer of complexity to the security, as attackers could exploit task preemption during this phase to manipulate the patching process before it is complete.

Goal 3: Tamper Resistance

Protect the context of the hotpatching framework and its runtime state during the hotpatch application process, ensuring secure application of hotpatches.

After Patching. Once a hotpatch has been successfully applied to the firmware, it is essential to preserve its integrity to ensure the continued effectiveness of the hotpatch. An attacker might attempt to tamper with the applied hotpatch code by overwriting it, modifying its functionality, thereby endangering the stability and security of the running system. Furthermore, changes to associated metadata, such as state information or memory pointers, could be altered to disable or redirect the hotpatch logic, effectively invalidating the hotpatch. This could silently reintroduce the original vulnerability into the firmware, making the hotpatch ineffective while avoiding immediate detection.

Goal 4: Post-Application Integrity

Preserve the integrity and security of applied hotpatches and their metadata, preventing tampering or unauthorized modifications after application.

Formal Security Analysis. We have outlined the specific security goals that our framework aims to achieve. However, we must substantiate these claims to establish a trustworthy security foundation. To do so, we must model the framework, including how it handles hotpatches from reception to application, as well as after the patching process. Additionally, we must model its interactions with the RTOS kernel and memory. Finally, we must provide evidence proving the validity of our security claims about our framework.

Goal 5: Formal Security Analysis

Provide a formal security model and security reasoning to establish the secure-by-design approach of the framework. The evidence must validate the framework's ability to resist tampering and other identified threats, ensuring that the security claims made in its design are satisfied by the framework.

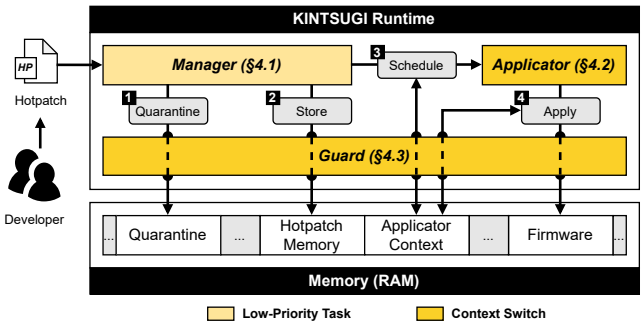


Figure 2: Overview of the design for KINTSUGI.

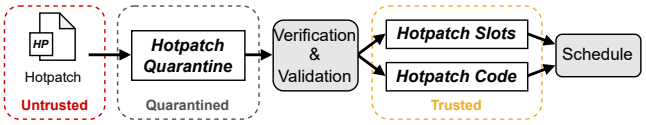


Figure 3: Worfklow of the *Manager*.

4 KINTSUGI

We address the above goals through the design of KINTSUGI, the first security-focused hotpatching framework for code-shadowing real-time embedded devices. KINTSUGI balances the timing constraints of the underlying RTOS with the need to protect the hotpatching process and hotpatches themselves from tampering attacks. Figure 2 gives an overview of KINTSUGI, showing its three core components: the *Manager*, the *Applicator* and the *Guard*. The *Manager* runs as a low-priority task within the RTOS, responsible for quarantining untrusted hotpatches for verification and validation 1. If approved, the hotpatch is stored in the trusted memory of KINTSUGI 2 and scheduled to the *Applicator* by configuring the applicator context 3. This context facilitates communication between the *Manager* and *Applicator*, which applies the hotpatch atomically to the firmware 4. Additionally, the *Guard* runs during the context switch, enforcing memory policies on predefined regions of KINTSUGI and the firmware to safeguard against unauthorized access and tampering attacks.

4.1 Manager

The *Manager* is responsible for both stored and incoming hotpatches. We illustrate its workflow in Figure 3. Hotpatches received by the device are considered untrusted, as they reside in memory that is not directly managed by KINTSUGI (i.e., heap-allocated). The *Manager* first moves hotpatches to a dedicated quarantine memory region fully controlled by KINTSUGI. Here they undergo verification and validation. If these sanity checks succeed, the hotpatch is considered trusted, addressing Goal 2. After this, hotpatches are segregated: non-executable metadata is stored in designated hotpatch slots, while executable code is allocated into hotpatch code.

Hotpatch Quarantine. Hotpatches, even when transmitted over a secure channel, must be treated as untrusted until verified. In an RTOS, however, verification and validation cannot safely occur in shared memory since a malicious task could exploit preemption to compromise a hotpatch during the verification phase, i.e., after the *Manager* checks it but before it is stored, causing a time-of-check to time-of-use (TOCTOU) vulnerability. To eliminate this risk, KINTSUGI introduces a *hotpatch quarantine*: an isolated, non-executable memory region under the *exclusive control* of the *Manager*. No other component can access the quarantine, even if the *Manager* is preempted during verification. This guarantees that a hotpatch cannot be altered once it is in quarantine. Once verified, the hotpatch is stored to a trusted memory region accessible to the rest of the framework. This strict separation is essential for both preventing runtime tampering and clearly enforcing trust boundaries, as detailed in Section 7.

Verification & Validation. To prevent tampering of the hotpatch, the *Manager* must verify the integrity of the hotpatch after it is stored in the quarantine. While we stress the importance of integrity checks, we do not specify a digital signature scheme, as this depends on the device's cryptographic capabilities. Nonetheless, robust integrity checks are mandatory before transferring the hotpatch to trusted memory.

After integrity verification, the *Manager* checks the hotpatch for malformed metadata, ensuring it includes, for instance, a valid target address in the firmware. This address must be validated against a predefined table of allowed memory regions to avoid overwriting volatile data, which can cause system crashes. The size of the hotpatch and its code must also be validated to prevent buffer overflows issues. Once these checks have passed, the hotpatch can be stored in trusted memory of KINTSUGI and subsequently becomes eligible for application to the firmware.

Hotpatch Slots & Code. Once a hotpatch has passed all verification and validation checks, it becomes ready to be stored in KINTSUGI's hotpatch memory. While traditional hotpatching approaches dynamically allocate memory on the heap to store hotpatches, KINTSUGI introduces two dedicated memory regions for secure storage and isolation of hotpatch metadata and code, as shown in Figure 4. The first area contains dedicated slots that store only the metadata of the hotpatch, which includes its status (i.e., pending, scheduled or applied), a unique identifier, the header of the hotpatch and a pointer to the hotpatch code in memory. The second area is reserved for the to-be executed code of the hotpatches. By rigorously separating metadata from executable code, KINTSUGI enforces robust data execution prevention, significantly reducing the attack surface. This is especially important in the context of code-shadowing devices, where firmware is mixed with volatile data, which increases the risk of data execution vulnerabilities. Once the hotpatch is stored in the slots and code memory regions, it can be applied by the *Applicator*, which is configured by the *Manager*.

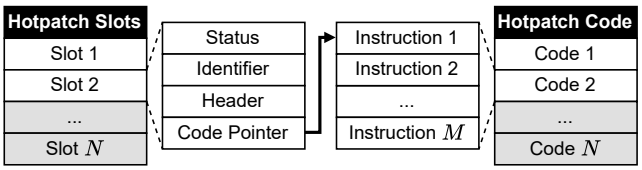


Figure 4: Layout of the hotpatch slots containing the metadata of the hotpatch and the hotpatch code which stores the executable code. The hotpatch slot contains a pointer to the dedicated hotpatch code in memory.

Hotpatch Types. Patching vulnerabilities requires the introduction of new code to the running firmware. We classify the types of hotpatches into two categories. First, the *redirect* type considers vulnerabilities that require the introduction of additional logic to, for instance, add new input validation checks, such as `CVE-2017-2784` [55] which disables the usage of the elliptic curve *secp224k1*. For these type of hotpatches, the original control flow of the firmware must be redirected to the hotpatch code stored in KINTSUGI's memory. Second, the *replacement* type directly overwrites code in memory to, for example, change the sign of a branch from signed to unsigned, preventing privilege escalation attacks, such as in `CVE-2020-10024` [56].

4.2 Applicator

The *Applicator* is a software-based unit responsible for actively applying hotpatches to the firmware. It is carefully designed to meet the goal of real-time integrity preserving, atomic application of hotpatches, as outlined in Goal 1. Additionally, the *Applicator* executes as a deterministic unit with as little complexity as possible. The *Applicator* is the *only* component with write access to the firmware image, making it the sole component authorized to apply hotpatches. The *Manager* can configure the *Applicator* through the application context, initiating the application process and receiving updates on its completion.

Atomic Hotpatch Application. As discussed in Goal 1, the atomic application of hotpatches to the firmware in code-shadowing real-time embedded devices, resulted in an intractable dependency problem. This is because we need a critical section to ensure atomicity. But we cannot include such a section in the *Manager* task, because it would stop the system execution unpredictably. Thus, we closely examined the execution model of RTOSes and identified a critical section in the *context switch* mechanism. This critical section ensures that the storage and restoration of task execution contexts are indivisible, and thus, *atomic* operations. Otherwise the system could enter a corrupted state if its execution is interrupted while saving or restoring the task's execution state in the registers. We therefore leverage this critical section to run the *Applicator*, ensuring atomic hotpatch application.

Configuration Registers. The *Applicator* features a set of registers stored in the applicator context memory. These registers are conceptually similar to memory mapped input-output (MMIO) registers from hardware units like the MPU and serve as the primary interface for configuring the *Applicator*. There are three registers that can be configured. First, the *status* which indicates whether a hotpatch is scheduled or not. Next, the *target address* to apply the hotpatch to, and last, the *hotpatch data* which contains the bytes to write to the target address. These registers are only writable by the *Manager* and the *Applicator*, acting as a communication link between the execution of a task and the context switch.

Application Procedure. The *Applicator* actively applies the hotpatch to the firmware by writing the hotpatch data to the target address from its configuration registers in the applicator context, only and if only the status is set to *active*. To reduce the impact of the overhead introduced by the *Applicator*, it only applies the hotpatch, if the *next task to execute* is the *Manager*. This prevents higher priority tasks unrelated to KINTSUGI from executing without delay, while isolating the overhead to a controlled execution context where it can be better managed. Once this condition is met, the *Applicator* takes the information from its configuration register and writes the data to the target address as an indivisible, constant-time operation. Finally, it updates the status indicating that the hotpatch is applied, notifying the *Manager* in preparation for the next execution cycle.

4.3 Guard

Preventing unauthorized access to the framework, hotpatches, or the firmware itself is a key design principle of KINTSUGI. This is achieved through the use of the MPU, a standard feature in modern MCUs, which allows precise control over memory access. By configuring the MPU to dynamically grant or revoke access rights based on the current execution state, we ensure protection for the memory regions of KINTSUGI. When a memory policy is active, and an access to a protected memory region occurs (i.e., writing to it), an exception is triggered. In KINTSUGI, we let the exception act as a pass-through, resulting in no effect, thus leaving the memory unchanged. To incorporate this mechanism into KINTSUGI, we have designed the *Guard*, which enforces memory policies across the hotpatch quarantine, slots, code, context as well as on the firmware itself. Through this, the *Guard* addresses Goal 2 and Goal 3, as it prevents attackers from tampering with the hotpatching quarantine and context. Additionally, it fulfills Goal 4 by safeguarding the integrity of the hotpatch memory against unauthorized modifications.

Memory Policies. We summarize the memory policies enforced by the *Guard* in Table 1. The primary goal of these policies is to minimize write access permissions by default. For instance, only the firmware image and the hotpatch code are allowed to execute, while all other regions are marked as

Table 1: Memory policies enforced by the *Guard*.

Component	Firmware (X)	Context (XN)	Slots (XN)	Quarantine (XN)	Code (X)
Default	RO	RO	RO	RO	RO
Manager	RO	RW	RW	RW	RW
Applicator	RW	RW	RO	RO	RO

X: Execute, XN: Execute Never, RO: Read-Only, RW: Read-Write

XN, preventing data execution attacks. By default, all memory regions are configured as read-only unless explicitly required by a component of KINTSUGI. This default applies when neither the *Manager* or *Applicator* are executing. As a result, even if an attacker gains control over a task unrelated to KINTSUGI, they are prevented from modifying critical memory regions, such as the firmware image or hotpatch code, thus ensuring the system's integrity. During the execution of the Manager, it requires access to the quarantine to temporarily store incoming hotpatches, to the slots for permanent hotpatch storage, and to the context for scheduling hotpatches. Similarly, during its execution, the *Applicator* requires write access to the firmware image to apply hotpatches and to the context to update its status.

Memory Policy Enforcement. To enforce these policies, the *Guard* runs in privileged mode in the RTOS kernel, executing alongside the *Applicator* during context switches. While this placement ensures tight control over task transitions, it also introduces additional overhead into the scheduling process. Therefore, we highly optimize the *Guard* to have minimal impact on overall system performance.

The functionality of the *Guard* is structured as an efficient state machine, as described in Algorithm 1. This state machine adjusts access rights based on whether the *Manager* has executed before (M_P) or will execute next (M_N). The execution of the *Applicator* is encapsulated by the *Guard* because it requires access to both the firmware and the context, so the *Guard* manages its execution by granting and revoking write access as needed. A detailed explanation of the algorithm's functionality and its optimization for minimal performance impact is provided in Appendix A.

5 Implementation

We implement a secure hotpatching solution by creating a prototype for KINTSUGI for the popular RTOSes FreeRTOS [7] and Zephyr [15]. KINTSUGI consists of 1745 lines of C-code, 142 lines for linker-script definitions and 892 lines of python code to generate hotpatches. To evaluate KINTSUGI on real hardware, we utilize the `NRF52840-DK` with 1 MB of flash and 256 kB of RAM. The `NRF52840` contains the widely used ARM Cortex-M4 MCU, a processor used in millions of devices. This MCU runs at 64 MHz and has an MPU with up to 8 configurable memory regions.

Algorithm 1 Pseudocode of the *Guard* functionality.

Input: M_P, M_N

```
1:  if M_N == 1 then
2:      grant_access(applicator_context);
3:      if hotpatch_scheduled() == 1 then
4:          grant_access(firmware);
5:          Applicator();
6:          revoke_access(firmware);
7:      end if
8:      grant_access(hotpatch_slots);
9:      grant_access(hotpatch_code);
10: else if M_P == 1 then
11:     revoke_access(applicator_context);
12:     revoke_access(hotpatch_slots);
13:     revoke_access(hotpatch_code);
14: end if
```

RTOS Instrumentation. To integrate KINTSUGI into an an RTOS, some modifications are needed for the context switch to invoke the *Guard*. Specifically, we need to include a call to the *Guard*, i.e., the `hp_guard_applicator` function, in the `vTaskSwitchContext` function for FreeRTOS and in `z_arm_pendsv_c` for Zephyr. Since the *Guard* needs to know about the previous and the next task that was and will be run, we extend task creation in FreeRTOS by adding a new attribute which identifies the *Manager* task to the task control block (TCB), a structure associated with each task running on the RTOS that holds all task-related information. We then use this attribute and pass it to the *Guard*. In FreeRTOS, we also have to modify the task creation function to check if the new task is the *Manager*. We accomplish this by a simple check on the task name, making sure that the *Manager* has a unique task name. On the other hand, in Zephyr, we can directly access the previously executed task from its context switch and pass the information directly to the *Guard*.

Manager. We implement the *Manager* as a low-priority task that awaits incoming hotpatches to be processed, stored and scheduled. Since we use our own defined memory regions, the *Manager* introduces custom allocation methods. The hotpatch slots are arranged as a memory pool for a fixed amount of hotpatch entries defined by the developers in a configuration file. For each incoming hotpatch the *Manager* selects one of the free slots. Memory allocation for the hotpatch code is performed through a free-list memory allocator similar to `malloc`, ensuring the *Manager* can allocate hotpatches of varying sizes, as some require only 8 bytes while others require more than 100 bytes.

Code Detouring. Hotpatches of type *redirect* are implemented through a code detouring mechanism inspired by Detours [26]. Since we do not make any strong assumptions about the memory layout, and thus do not expect that certain memory regions are placed at specific addresses, we require a solution that works arbitrary of its position and is able to

address the *complete* memory space, if necessary. We accomplish this, by utilizing static code blocks of 8 bytes that consist of a pc-relative load that overwrites the pc-register itself, as well as a target address to branch to, i.e., the hotpatch code:

```
<hotpatch_target+0>: ldr pc, [pc, #0]
<hotpatch_target+4>: <hotpatch_code>
```

During execution, the *ldr* instruction will load the 32-bit word that resides *after* the current instruction (thus `hotpatch_target+4`) into the pc-register itself, acting as a branch that can address all possible 32-bit addresses. We want to note that, depending on the address of `hotpatch_target`, one must adjust the alignment within the *ldr* instruction, resulting in the immediate offset which gets added to the pc-register to be 2 instead of 0, which KINTSUGI takes care of.

We also confirm that this code detour is supported by 30 out of 33 architectures supported by the ARM GCC compiler toolchain, with exception of `ARMv6-M`, `ARMv6S-M`, and `ARMv8-M.Base`. This result is based on an experiment that we conducted, and the code is included in our artifact.

Guard & Applicator. We implement the policy enforcement of memory regions using the memory-mapped registers to access the MPU of the MCU [1]. The `grant_access` and `revoke_access` functions of the *Guard* configure the MPU applying active or inactive configurations for each memory policy. Active memory regions grant the dedicated component read-write access while inactive regions have read-only access. To reduce overhead, the *Applicator* logic is inlined into the *Guard*, eliminating the need for the *Applicator* to check if a hotpatch is scheduled. The *Applicator* only requires 7 inline assembly instructions to read from the applicator context and write the data to the target address.

Hotpatch Development & Generation. To create effective hotpatches for KINTSUGI, we have developed a streamlined system for patch development and generation. These hotpatches are written in C and integrated into the firmware source code. We implement a set of helper functions to simplify the hotpatch development process. During compilation, the object files for the hotpatches are generated but omitted in the final firmware binary. Instead, we extract the hotpatch bytecode from these files and modify it to fit the dynamic firmware image by replacing symbols with their addresses. This process is automated using a Python script that assembles the hotpatch header and code. We show an example of a hotpatch addressing `CVE-2017-2478` [55] in Appendix B.

6 Performance Evaluation

We first evaluate the performance of KINTSUGI, conducting micro-benchmarking and evaluating the memory footprint. After that, we perform security experiments followed by evaluating KINTSUGI in real-world settings by hotpatching actual CVEs and performing a case study.

Experimental Setup. In our performance evaluation, we use hotpatch sizes based on `RapidPatch` [40] and `AutoPatch` [62], extended by the average across these sizes, respectively. For experiments with multiple hotpatches, we use a count of 64, similar to `RapidPatch`, increasing by steps of 4. Each experiment was performed 100 times to obtain an average result.

6.1 Micro-Benchmarking

Measuring the performance of security components in RTOSes is a challenging task due to the diversity of real-time applications. Accurately assessing the overhead introduced by a framework operating in an RTOS requires thorough analysis of its components through micro-benchmarking. Note that we do not perform experiments to assess the overall system impact of KINTSUGI on a specific system. This choice is motivated the wide variety of real-time applications. Our primary focus is on analyzing the overhead, particularly regarding its effect on the context switching within the RTOS.

For the micro-benchmarking experiments, we measure performance using the high-precision timers of the ARM Cortex-M4 processor. To obtain accurate measurements for each component, we execute the experiments over 100 iterations and take the average. We simulate real-world conditions of the hotpatch application by triggering the hotpatch processing through an interrupt that notifies the Manager task every second to ensure cold measurement results.

Manager. In the first experiment, we measure the performance of the *Manager*. In particular, we evaluate the execution time to validate, store and schedule incoming hotpatches as a whole and individually. For this evaluation, we omit the KINTSUGI verification of hotpatches, as it is highly dependent on the implemented cryptographic signature scheme. Nevertheless, we will discuss potential candidates for the verification and validation of hotpatches in Section 11.

We conduct the experiment by setting up a task that generates hotpatches of various code sizes. These hotpatches are then passed to the manager, and the execution time of the component of interest is measured. We present the results of the micro-benchmarking evaluation in Table 2, showing the measurements from the *minimum*, *average* and *maximum* hotpatch sizes from `RapidPatch` and `AutoPatch`, respectively. Here, we only show a subset of the results and refer to Table 5 in Appendix D, which includes the complete set of hotpatch sizes from both works. The validation and scheduling of hotpatches require the fewest cycles—47 cycles (0.73μs) and 117 to 118 cycles (1.84μs and 1.84μs), respectively—regardless of hotpatch code size, demonstrating that these steps operate independently of input size and with high efficiency. The increase in execution time is primarily due to the fact that hotpatch storage is input size dependent, meaning that larger hotpatches take longer to process. It is also important to note that the table does not explicitly include the time required to

Table 2: Micro-benchmarking results of the *Manager* for hotpatch sizes from RapidPatch (RP) [40] and AutoPatch (AP) [62].

	Hotpatch Size (Bytes)	Manager Cycles	Manager Time (μs)	Validation Cycles	Validation Time (μs)	Storing Cycles	Storing Time (μs)	Scheduling Cycles	Scheduling Time (μs)
RP	48	2507	39.17	47	0.73	874	13.66	118	1.84
	131	5094	79.59	47	0.73	1718	26.84	118	1.84
	260	9092	142.06	47	0.73	3006	46.97	117	1.83
AP	380	12812	200.19	47	0.73	4206	65.72	117	1.83
	548	18020	281.56	47	0.73	5886	91.97	117	1.83
	944	30281	473.14	47	0.73	9831	153.61	117	1.83

copy the hotpatch to the quarantine zone, as the process is implemented through memory copy operations. This action occurs very early in the execution of the *Manager*, so the sum of the sub-components does not add up to the total execution time of the *Manager*.

Guard & Applicator. We evaluate the performance of the *Guard* and *Applicator* by analyzing the overhead induced during context switching. This evaluation is based on the specific conditions derived from Algorithm 1 of the *Guard*, namely M_P, M_N, and whether a hotpatch is scheduled or not. We present the results in Table 3, where we show the execution times in cycles and microseconds for each condition of the *Guard*. The most common condition to execute is $\neg M_P \wedge \neg M_N$, which indicates that the *Manager* is not executed before or after the current task. This is due to the low priority of the *Manager* and its limited role in the overall system. We observe that the constant overhead introduced by KINTSUGI during the context switch is only 38 cycles (0.59μs), a negligible amount. Furthermore, in cases where the *Manager* has been executed previously we measure 127 cycles (1.98μs). The highest overhead occurs when the next task is the *Manager*, due to the access-granting process. In cases where no hotpatch is scheduled, this process adds only 133 cycles (2.08μs), which is slightly more than the previous condition. If a hotpatch is scheduled, the combined execution time for the *Guard* and *Applicator* is 216 cycles (3.38μs). Note that the condition with the highest overhead is also the one that occurs least frequently. This is because the event of applying a hotpatch in relation to the number of context switches is extremely low.

6.2 Hotpatching Scalability

To assess KINTSUGI's scalability in terms of performance, we conducted an experiment in which we applied multiple consecutive hotpatches and measured the execution time encompassing processing, scheduling, and applying hotpatches across different hotpatch sizes and counts.

The measurements shown in Figure 5 visualize the execution time. As can be seen in the plot, the size of hotpatches has a negligible impact on the overall execution time, as indicated by the linear gradient with respect to the hotpatch count. This suggests linear scalability, implying that even a large amount of consecutive hotpatches does not degrade performance. This linear gradient results from measuring the full hotpatching process of KINTSUGI, meaning we invoke the *Manager* to install up to 64 hotpatches consecutively. Consequently, the impact of the memory copy operations, which contribute to the performance overhead of larger hotpatches, plays a minimal role in the overall performance of KINTSUGI. Precisely, our measurements show that applying a single hotpatch from start to finish takes around 0.9*ms*, 15.38*ms* for 16 hotpatches, 30.87*ms* for 32 hotpatches and 61.75*ms* for up to 64. We therefore conclude that KINTSUGI scales in its performance scalability linear to the number of hotpatches it applies consecutively, where the size of hotpatches only plays an insignificant factor.

We also emphasize that this experiment demonstrates the *robustness* of KINTSUGI in terms of runtime behavior, as hotpatches are not only applied but also successfully removed in every iteration, across all tested hotpatch counts and sizes.

Table 3: Measurement results of the micro-benchmarking for the *Guard* and *Applicator*.

Component	M_P	M_N	Hotpatch Scheduled	Cycle Count	Time (in μs)
Guard	○	○	○	38	0.59
	●	○	○	127	1.98
	○	●	○	133	2.08
	○	○	●	38	0.59
	●	○	●	127	1.98
	○	●	●	216	3.38
Applicator	○	●	●	83	1.30

M_P = Previous task was the *Manager*, M_N = Next task is the *Manager*

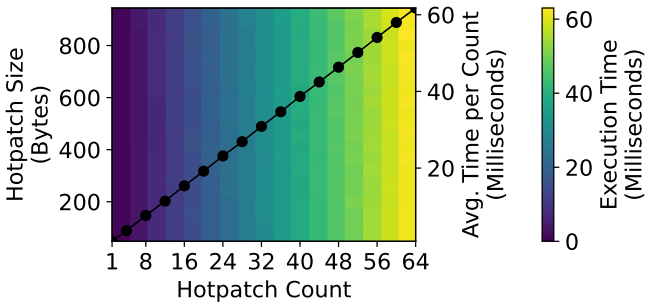


Figure 5: Performance measurement of processing multiple consecutive hotpatches in KINTSUGI.

6.3 Memory Overhead

In addition to evaluating KINTSUGI's performance, we assess its scalability by measuring the memory overhead introduced by varying hotpatch sizes and counts. This experiment provides insights KINTSUGI's resource usage across different deployment scenarios, with a particular focus on memory

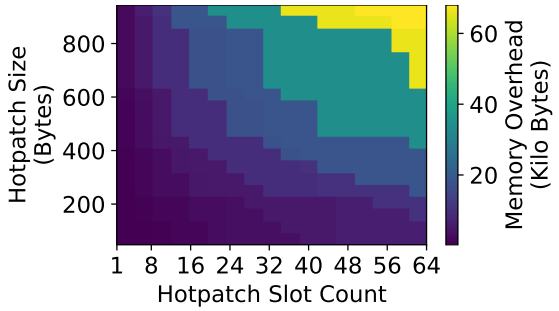


Figure 6: Resource utilization for varying number of maximum hotpatch sizes and slot counts.

requirements for storing hotpatch metadata and code.

To quantify memory overhead, we compile KINTSUGI with different hoptatch slot counts and sizes, extracting the memory consumed by four memory regions: *hotpatch_slots*, *hotpatch_quarantine*, *hotpatch_context* and *hotpatch_code*. The sum of these regions represent the total memory footprint of KINTSUGI in its configurable dimensions. Figure 6 shows how memory usage varies with these configuration parameters. We observe that increasing the number of hotpatch slots results in smaller overhead increases, whereas larger hotpatch sizes have a more substantial impact. For example, using sizes from `RapidPatch`, the minimum hotpatch sizes of 48 Bytes yields an average overhead of 3.50*KB*; the average size of 131 Bytes results in 7.63*KB*; and the maximum of 260 Bytes leads to 13.33*KB*, each measured across the 64 possible slots. In contrast, `AutoPatch` incurs 19.3*KB* for its minimum size of 380 Bytes per hotpatch, 27.38*KB* for the average of 548 Bytes, and 42*KB* for the maximum size of 944 Bytes.

The stepped pattern in the plot reflects memory alignment constraints required by the MPU configuration. Specifically, *hotpatch_slots* and *hotpatch_quarantine* share an MPU region and are aligned together, which results in plateau regions between memory usage increases.

6.4 Hotpatching Real-World CVEs

To demonstrate KINTSUGI's applicability in hotpatching real-world vulnerabilities, we conduct experiments on 10 CVEs. This dataset was compiled by RapidPatch [40][2] and includes a diverse range of vulnerabilities. With this collection we address vulnerabilities in both FreeRTOS [7] and Zephyr [15] RTOSes, libraries such as mbedTLS [10], and the AMNESIA:33 [16] vulnerabilities, in particular two that affect picotcp [14]. We present a list of the vulnerabilities along with resulting hotpatch sizes in Table 4 and refer the reader to Table 6 in Appendix E for the full table containing measurements from the micro-benchmarks. Although the development and secure generation of hotpatches are separate from our goals, we focus on the safe and efficient application of hotpatches to demonstrate that KINTSUGI can effectively address real-world vulnerabilities.

[2]Two CVEs were excluded due to compatibility issues with the MPU and deprecated firmware.

Table 4: CVEs implemented and mitigated by KINTSUGI.

CVE-ID	Component	Type	Hotpatch Size (Bytes)
CVE-2020-10021	ZephyrOS	Redirect	104
CVE-2020-10023	ZephyrOS	Redirect	40
CVE-2020-10024	ZephyrOS	Replacement	24
CVE-2020-10062	ZephyrOS	Redirect	144
CVE-2020-10063	ZephyrOS	Redirect	122
CVE-2018-16524	FreeRTOS	Redirect	100
CVE-2018-16603	FreeRTOS	Redirect	60
CVE-2017-2784	mbedTLS	Redirect	64
CVE-2020-17443	AMNESIA33	Redirect	56
CVE-2020-17445	AMNESIA33	Redirect	76

For each CVE, we developed test cases that exploit the vulnerability and corresponding hotpatches. We *successfully* hotpatched all vulnerabilities. Our workflow involves creating a vulnerable task that triggers the vulnerability while executing compromised functions and printing results via UART. After validating and scheduling the hotpatch with the *Manager*, the *Applicator* applies it to the firmware. We observe that the vulnerability has been successfully patched, as the firmware now outputs the correct values.

7 Security Analysis

In this section we provide a formal security analysis of KINTSUGI to achieve Goal 5. Note that Goal 1 is not related to the security of hotpatching, but to the *correctness* of the kernel, and is therefore considered out of scope.

Assumptions. KINTSUGI extends RTOS kernels with secure hotpatching capabilities. While verifying the correctness and security of an entire kernel is a complex and independent challenge, our focus is on demonstrating that, given a reasonably implemented kernel, our extensions *preserve* its security properties and *enhance* it with hotpatch support. We assume the hardware capabilities from Section 3.1, where the kernel has full and exclusive control of an MPU to establish disjoint memory regions ρ and manage their permissions. Other than this, we only rely on standard RTOS properties like scheduling, context switching and resource management.

Execution Semantics and Threat Model. To reason about KINTSUGI and its interactions with the kernel and the attacker, we need to model these three components. First is KINTSUGI*'s code* itself, modeled faithfully as the pseudocode-like language from, e.g., Algorithm 1, which forms a standard program logic. We use two semantics to model execution. The *Applicator* and the *Guard* are part of the kernel and use a standard, sequential semantics $\rightarrow_{seq}$ that

sees the full behavior of the machine: when an instruction ι executes from a state σ, the next state σ' depends only on ι.

In contrast, the *Manager* runs as unprivileged code, so its execution can be interleaved with that of *kernel and attacker code*, the other two components. However, we do not need to model these explicitly. Instead, to overapproximate their effects on user code, we use a second, preemptive semantics $\rightarrow_{pre}$ which builds in their interference: when an instruction ι executes from a state σ, to account for possible interruptions and unforeseen side effects, we assume the next state σ' *may* have suffered arbitrary changes, *except* for those regions of memory ρ where the task has exclusive write access, which depend only on ι.

The preemptive semantics $\rightarrow_{pre}$ for a user task needs to be adequate with respect to the sequential semantics $\rightarrow_{seq}$ running all system code. We cannot prove adequacy without verifying the kernel itself, but this is indeed a property we expect from any reasonable RTOS. The resulting attacker model is quite strong, which would simplify this verification, and emphasizes the local nature of reasoning on unprivileged code, which can only reliably depend on its own local memory.

Verification & Validation (Goal 2). This goal is technically the most involved, as it takes place entirely within the unprivileged *Manager*, but verification is fairly simple. The code operates on three regions of memory: an input buffer for hotpatches ρ_{ib}, a special quarantine zone ρ_{qz}, and KINTSUGI's hotpatching state ρ_{hs} (including hotpatch slots, code and firmware memory, and control flags for the *Applicator*). By construction ρ_{qz} belongs to the *Manager*; by assumption ρ_{hs} belongs to the kernel, which lends it to the *Manager* during its execution; and ρ_{ib} is shared and potentially under control of the attacker. Only authenticated hotpatches may be applied to the system. Let π denote a hotpatch received on the input ρ_{ib}, so $\rho_{ib} \mapsto \pi$. By assumption, we model the behavior of the verification and validation functions such that $validate(\rho)$ and $verify(\rho)$ succeed, iff the patch $\rho \mapsto \pi$ satisfies authenticity, integrity and validity. Dually, if the patch is invalid *or becomes corrupted*, either or both will return false.

The code of the *Manager* κ_M is divided into two phases. The first phase, say κ_M^1, copies an incoming hotpatch from ρ_{ib} to the private ρ_{qz}. The second phase, say κ_M^2, performs the sanity checks on ρ_{qz} and decides whether to apply or discard the hotpatch. We need to show that if we run $(\kappa_M, s) \rightarrow_{pre}^* s'$, where ρ_{ib} contains a patch π in the initial state s, the execution succeeds iff π is well-formed and has not been tampered with.

Proof Outline. The execution of the first phase is represented by a preemptive execution $(\kappa_M^1, s) \rightarrow_{pre}^* s'$, where the initial state s contains the patch, $\rho_{ib} \mapsto \pi$. By unrestricted preemptive execution on shared resources, the contents of ρ_{ib} could have changed arbitrarily. By case analysis on the contents of $\rho_{qz} \mapsto \pi'$ in s', we have two possibilities. If $\pi = \pi'$, the copy succeeded, and by the specification of $validate(\rho_{qz})$ and $verify(\rho_{qz})$ both return true and the application of the hotpatch can proceed. On the contrary, if $\pi \neq \pi'$, $validate(\rho_{qz})$ and $verify(\rho_{qz})$ detect the changes and abort application.

Once the hotpatch is declared safe, its contents are copied from ρ_{qz} to ρ_{hs}, and finally the *Applicator* receives the green light from the *Manager* (this execution of κ_M^2 only uses private memory, so it is indistinguishable from sequential execution). This is the only possible way to invoke the *Applicator*.

Tamper Resistance (Goal 3). This is an integrity property on the configuration data memory, which is part of ρ_{hs}. The relevant parts of ρ_{hs} must be protected from the attacker while the patch is being applied, i.e., during the execution of the *Applicator*. We need to show that whenever the *Applicator* executes, i.e., its code κ_A runs as an execution of the form $(\kappa_A, s) \rightarrow_{seq} s'$, the attacker has no influence on this process.

Proof Outline. The *Applicator* runs sequentially as part of the critical section and thus is not subject to interruptions. It suffices to show that the applicator *only* runs when *safe* hotpatches are sent to it through ρ_{hs}. By assumption, the attacker is unable to directly compromise the kernel: it can only attempt to corrupt hotpatch candidates. The memory footprint of the *Applicator* only involves memory in ρ_{hs}, so it suffices to show that attacker data cannot reach this region. By assumption, the *Guard* is the sole administrator of ρ_{hs}, and it is easy to see that it only shares it with is the *Manager*: the problem reduces to ruling out interference from this source. This follows as a consequence of Goal 2, which shows that the *Manager* only transfers uncorrupted data to ρ_{hs}, and the *Applicator*'s "go" flag is only set to true by *Manager* at the end of a successful hotpatch transfer, so an execution $(\kappa_A, s) \rightarrow_{seq} s'$ can only take place when uncorrupted hotpatch data is in ρ_{hs}.

This result can easily be extended from the execution of the *Applicator* code itself to longer runs, precluding manipulations of its state during normal system operation.

Post-Application Integrity (Goal 4). This is another integrity goal which considers parts of ρ_{hs} to ensure the overall integrity of a hotpatch even after it has been applied. KINTSUGI tightly orchestrates the progression of hotpatch slots σ through several satuses: inactive, pending, scheduled, and finally active. This goal is concerned with the contents of a slot after it has been applied and its status becomes active. For an arbitrary execution $s \rightarrow_{seq}^* s'$, if σ is active in s, it remains so in s', and its corresponding contents in ρ_{hs} are unchanged between s and s'.

Proof Outline. The general security argument aligns closely with that of Goal 3. By analysis of the memory footprint of the code, only the critical section and the *Manager* task can ever access the contents of ρ_{hs}. Similarly, we can establish the workflow of hotpatch states, and the fact that the contents of active hotpatch slots are never modified, as well as the fact that the active state is the final state of hotpatch slots until the system is rebooted. As a consequence of previous goals we conclude hotpatch slots are never modified after application. Generalizing this result to support rollback of hotpatches and reuse of slots only affects the correctness of this code; the security analysis remains unchanged.

8 Security Experiments

We now demonstrate that our security claims hold true in practice. To this end, we conduct several experiments that address the three stages of hotpatching outlined in Section 3.4. In particular, we aim to practically demonstrate KINTSUGI's security *before*, *during* and *after* the hotpatching process.

For the experiments, we implement different test cases for each of the security goals Goal 2, Goal 3 and Goal 4. To validate the effectiveness of protection against the implemented attack scenarios, we log the output of the firmware over universal asynchronous receiver-transmitter (UART). As per our design, we let the exceptions generated by the MPU pass through (i.e., nothing happens). However, for demonstrating purposes we use the exception handler to acknowledge when the attack attempt fails. Each experiment considers the same attacker from our threat model (Section 3.1). Additionally, the attacker has knowledge about the resources of KINTSUGI (i.e., internal memory addresses such as the applicator context).

We realize this scenario by implementing the attacker as a static task that executes sequentially to the *Manager*. We further extend the test cases by adding more tasks to the firmware with different priorities, making the binary execute in a more natural way and increasing the number of context switches. In the following we describe how the malicious task tries to compromise KINTSUGI in each scenario.

Before Patching. We consider a hotpatch that is already in memory and gets passed to KINTSUGI to be applied. Specifically, the attacker attempts to modify the raw hotpatch by changing all of the fields in its header to invalid values. However, due to the validation checks implemented by KINTSUGI, any attempted modification will result in the rejection of the hotpatch. We confirm that for each of these hotpatches, KINTSUGI actively rejects them, preventing malformed hotpatches from entering KINTSUGI's trusted environment.

During Patching. In this experiment, the attacker aims to modify the applicator context to inject malicious data, which can result in undefined behavior or greater control of the system for the attacker. While a hotpatch is scheduled and awaits its application by the *Applicator*, a context switch must occur in which the next task to execute is the *Manager*. During this time, there is a high probability that the attacker's task executes at least once before the next execution of the *Manager*. This creates a race condition as the attacker attempts to modify the applicator context. The attacker waits for the status of the applicator context to change to *scheduled* before altering its data. However, the *Guard* protects the memory region of the applicator context, ensuring that only the *Manager* and *Applicator* can make changes. Any unauthorized attempt to write to the applicator context from outside the *Manager* or *Applicator* results in a pass-through exception, confirming our security goal.

After Patching. The attacker monitors the *Applicator* during the application phase and retrieves metadata from the applicator context, including memory locations of the applied hotpatch code. Using this information, the attacker attempts to overwrite the hotpatch code with arbitrary values. However, due to the memory policy enforcement of the *Guard*, these write operations are blocked, triggering pass-through handling by the exception mechanism. The memory policies ensure that hotpatch code regions are write-protected immediately after application, leaving the attacker unable to tamper with the patch. As a result, we can confirm that an attacker is not able to modify the hotpatch post-application.

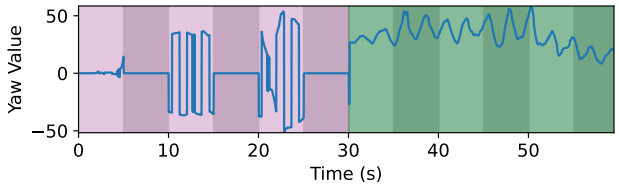


Figure 7: Yaw measurements from the drone over 60 seconds show toggling vulnerability every 5 seconds. The vulnerability is present for 30 seconds and then patched by KINTSUGI.

9 Case Study: Drone Stabilization

To perform a real-world case study, we evaluated KINTSUGI on the Crazyflie 2.0 drone [4], which operates on FreeRTOS version 10.4.0 hand handles multiple tasks, including maintaining stability through sensor data processing. Our case study focuses on the stabilization estimator, where we induced a vulnerability that toggles every 5 seconds, setting the yaw value to zero when active. The goal of KINTSUGI is to mitigate this issue by hotpatching the toggling code.

We assessed the impact of KINTSUGI with a Python script that logs the yaw value of the drone every 10*ms*, noting when the vulnerability is active and when the hotpatch gets applied. The experiment ran for 60 seconds, and our measurement results are illustrated in Figure 7. The shaded areas show when the vulnerability is active for 5 seconds, pulling the yaw value to zero. After the 30 second mark, KINTSUGI hotpatches the vulnerability and we can clearly see that the drone is able to stabilize itself again. This demonstrates that KINTSUGI can be effectively implemented into real-world applications.

In the shaded areas before the 30-second mark, we clearly see that the yaw value is pulled to zero and remains there for 5 seconds. However, after the 30-second mark, once KINTSUGI has applied the hotpatch, the drone successfully resumes estimating the yaw value, even though the vulnerability remains active in the shaded areas. This demonstrates that KINTSUGI can be effectively integrated and utilized in real-life products.

Performance Impact. We aim to evaluate the performance impact of KINTSUGI on the drone, specifically regarding context switches introduced by the *Guard* and *Applicator*. We

log the execution time and total number of context switches over 60 seconds, both with and without KINTSUGI enabled. This measurement is repeated 10 times to calculate the average cycle count. The results indicate that executing without KINTSUGI involves about 188 cycles per context switch, while enabling KINTSUGI increases this to 256 cycles, adding an overhead of roughly 68 cycles. However, the drone shows no signs of malfunctioning.

10 Related Work

Hotpatching. Before focusing on real-time embedded devices, the research community previously proposed several hotpatching solutions for other types of systems, such as Linux [21,28,36,41,60,70] and Android [29,30,73]. However, we cannot employ these methods into real-time embedded devices, which we consider in our work. Hotpatching in real-time embedded devices is an active research area, tackling the challenge of maintaining strict timing constraints during the hotpatching process. `HERA` [53] is, to the best of our knowledge, the first hotpatching framework targeting embedded hard real-time systems. The key idea is to utilize the flash-patch and breakpoint unit (FPBU), a hardware debug unit found in ARM Cortex-M3/M4 processors, to apply hotpatches to firmware executing from flash memory atomically. `RapidPatch` [40] uses extended berkeley packet filter (eBPF) to achieve architecture-independent deployment of hotpatches at the cost of runtime overhead. Further, `AutoPatch` [62] automatically generates hotpatches independently of the architecture using LLVM [8]. However, they do not consider code-shadowing devices and the security of the hotpatching process itself. In particular, they do not perform a security analysis on their approach or investigate potential attack vectors that an attacker might attempt to exploit in order to misuse their proposed framework. Additionally, there has been further research focusing on event-driven real-time applications through `Retcon` [71] and targeting internal control systems (ICSes) through `ICSPatch` [59] and `RLPatch` [76]. However, these types of real-time systems are outside of the scope of this work.

Security in Patching. The work of Wu et al. [72] investigated the overall security of firmware updates and aimed to identify common vulnerabilities in the updating process. Further, Nilsson et al. [54] and Asokan et al. [22] considered the security of the protocols and end-to-end security to deliver firmware updates. Recently, `FortifyPatch` [74] was introduced to enhance the security of the patching process on the device. However, `FortifyPatch` specifically targets cloud systems operating real-time Linux and leverages ARM TrustZone to achieve their security goals. While its objectives are similar to those of KINTSUGI, the targeted application environment is fundamentally different from ours.

Memory Isolation. Several frameworks have been proposed to enforce memory access control on embedded devices using a MPU. `EPOXY` [33] and `ACES` [32] partition the firmware into compartments with memory policies and instrument them using the LLVM compiler toolchain. `TyTan` [25] and `TrustLite` [48] propose and build on a custom architecture for a MPU using an FPGA, called the execution-aware MPU (EA-MPU). In addition, Hardin et al. [39] consider low-power MPU using the `MSP430` MPU. `OPEC` [77] focuses primarily on bare-metal firmware without a dedicated RTOS, and `RIMI` [45] introduces a memory access control approach for the RISC-V architecture. `Minion` [44] performs memory isolation from RTOSes by switching memory access control for each task. They cluster memory policies and enforce these during context switching, but their focus on the entire firmware differs from our goal of isolating the hotpatching process from the firmware. However, these approaches are not suitable for our specific case.

Code Detouring. Dynamic instrumentation of binaries through trampolines for code detouring is well-established, with earlier work from `Detours` [26], `Etch` [61], and `Dyninst` [27]. More recently, `Piston` [63] achieved this by inserting direct jumps for execution flow redirection. Static instrumentation has seen contributions like `Shimware` [38], `Dispatch` [46] and `RevARM` [47], which also employ direct branches, while `BiRFIA` [51] utilizes a combination of instructions for branching. Compared to these, KINTSUGI adopts the code detouring concept from `Detours`, implementing a fixed 8-byte trampoline that includes a pc-relative load and branch, providing coverage of the entire memory space.

11 Discussion

Comparison with State of the Art. In the following, we qualitatively compare KINTSUGI with prior work: `HERA` [53], `RapidPatch` [40] and `AutoPatch` [62]. While all target hotpatching for real-time embedded systems, their goals and approaches vary. Table 7 (Appendix F) outlines key differences in goals, assumptions, and methodologies, as well as support for our security goals outlined in Goal 2, Goal 3 and Goal 4. `HERA`, pioneered hotpatching for real-time embedded systems, focusing on instruction-level hotpatching in flash memory via hardware trampolines and a dispatcher.

`RapidPatch` extends this using eBPF virtual machines (VMs) for function-level hotpatching across different architectures, ensuring safe hotpatch execution but at a performance cost. `AutoPatch` automates hotpatch generation and applies weakest-precondition reasoning for safety but remains at function granularity. In contrast, KINTSUGI adopts a novel approach by providing instruction-level hotpatching for code-shadowing firmware, enabling direct code overwriting and code detouring without a dispatcher and support for full 32-bit addressability through trampolines. While it requires hardware support (like `HERA`), KINTSUGI is distinct in its *explicit* integration of security guarantees — before, during and after hotpatching — thereby addressing gaps such as runtime

tamper resistance and post-application hotpatch integrity not covered by previous work.

Generalization to Other Hardware Architectures. To integrate KINTSUGI into hardware other than the ARM Cortex-M series, the MCU needs to support hardware privilege separation and memory access control management. This capability is typically achieved through the use of an MPU. For example, the ARM Cortex-M [19] and Cortex-R [20] series of microcontrollers support an MPU, with the exception of the Cortex-M0 and Cortex-M1. Given the widespread use of ARM MCUs, we believe that KINTSUGI can be effectively implemented in a variety of devices. Porting KINTSUGI to other instruction set architectures (ISAs) is also possible, primarily depending on the implementation of the *redirect* hotpatch instrumentation. For instance, in the RISC-V architecture, one can implement the trampoline using absolute addressing and branching [12]. Therefore, the ability to construct a trampoline that the *Applicator* can apply to the firmware is essential, but this process can be leveraged by the *Manager*.

Adaptation to Other RTOSes. We have shown that KINTSUGI can be integrated into FreeRTOS and Zephyr. Adapting KINTSUGI to other RTOSes is also possible and requires modifying the context switch. However, since the context switch is an established procedure in RTOSes, we think it should be no problem to adapt KINTSUGI. For example, to integrate KINTSUGI into ChibiOS [3], a RTOS used in the Ardupilot [9] projects, one needs to integrate the *Guard* into the `PendSV` handler. By design, we aimed to minimize the required measures to integrate KINTSUGI into the RTOS.

Hotpatch Verification & Validation. As we pointed out in Section 4.1, we do not consider a precise digital signature scheme to be deployed together with KINTSUGI, as this is highly specific to the available resources. At this point, we want to refer to research [50, 52, 75], which consider the overall secure deployment of firmware updates, including schemes to perform verification on the firmware. Additionally, we emphasize that the validation of hotpatches, particularly of their metadata, must guarantee that no memory-related issues, such as buffer overflows, can occur. However, this requires proper input validation techniques such as code size boundary checks and validating address ranges for the hotpatch, which is the device maintainer's responsibility to implement carefully.

Schedulability Analysis & System Impact. While we measure the runtime overhead of KINTSUGI through microbenchmarking (Section 6.1) and scalability experiments (Section 6.2), we have not studied the schedulability of KINTSUGI. Our claims about real-time are based on the experimental results of our case study in Section 9. Precisely, we want to point out that for specific real-world use-cases, a schedulability analysis with KINTSUGI must be performed to guarantee that it meets the underlying systems real-time requirements. Furthermore, we would like to point out that the performance measurements in Section 6.1 represent raw overhead measurements and do not reflect the potential impact on a a real system. Reasoning about the total impact of KINTSUGI requires contextual reasoning about the underlying system into which our framework is integrated.

Limitations & Future Work. We first point out that KINTSUGI is bound to the number of memory regions by the MPU. Our prototype can implement KINTSUGI with four regions, as the slots and quarantine can share a region due to their overlapping policies. However, some RTOSes with MPU support may already use many regions, requiring further considerations when integrating KINTSUGI. Minimizing the number of regions is another subject of further exploration. For instance, Kim et al. [44] propose a *memory view clustering* technique to reduce required memory regions for memory isolation. Furthermore, while KINTSUGI rejects malformed hotpatches through the *Manager*, an adversary may inject malicious data before the hotpatch reaches it. This results in the *Manager* rejecting the hotpatch, causing a denial of service and preventing the application of hotpatches. To prevent this, further security measures are needed to mitigate this, but they fall outside of the scope of this work. To provide even more comprehensive evidence of the security of KINTSUGI, it would be interesting to develop a fully mechanized version of the security analysis in a separation logic framework like Iris [42], which could in principle connect the security proofs to a verified implementation of our tool.

12 Conclusion

This work presents KINTSUGI, the first hotpatching framework for code-shadowing real-time embedded devices, with focus on secure hotpatching. We focus on the balance between security and real-time applicability by leveraging the context switch of the RTOS for atomic hotpatch applications and to enforce strict memory policies. KINTSUGI shows minimal performance overhead ranging from 38 cycles (0.59μs) to a maximum of 216 cycles (3.38μs). We also formally specify KINTSUGI and provide a security analysis, demonstrating it resilience against various attack scenarios. To evaluate KINTSUGI's effectiveness, we implemented hotpatches for 10 CVEs, showcasing adaptability to real-world vulnerabilities. Finally, we integrate KINTSUGI into a drone running FreeRTOS, effectively applying hotpatches in a real-time.

Acknowledgments

This work was supported by the RWTÜV Foundation (project number: S0189/10037/2021), Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) as part of Priority Programme RAINCOAT II (440059533), and DFG under Germany's Excellence Strategy – EXC 2092 CASA – 390781972. We thank the reviewers and shepherd for their constructive feedback, and Felix Kalinowski, Sebastian Surminsiki and Agata Stanczyk for insightful discussions.

Ethics Considerations

We confirm that we have read the ethics considerations of USENIX'25 and fully support it. Our research aims to introduce a defense mechanism into the hotpatch application process for real-time embedded devices using code-shadowing. Our initial investigation showed that the memory associated with the hotpatching framework is particularly vulnerable to tampering attacks. In response, we developed a framework to protect against these attacks. All experiments for our evaluation were conducted in a controlled environment, without involving users or real-world systems. However, deploying our work in real applications necessitates rigorous firmware testing, including schedulability and response-time analysis. While we provide measurement results from our prototype, these values may vary in different environments and implementations and should be considered before deployment.

Open Science

We fully support USENIX's open science policy and comply accordingly. We make our artifacts, including the prototype implementation of KINTSUGI, details on RTOS integration, and the necessary source code files to reproduce our experiments, publicly available at https://doi.org/10.5281/zenodo.15592036.

References

[1] Arm Cortex-M4 Processor Technical Reference Manual Revision r0p1. https://developer.arm.com/documentation/100166/0001/Memory-Protection-Unit/About-the-MPU.

[2] The Arm Ecosystem Ships a Record 6.7 Billion Arm-based Chips in a Single Quarter. https://newsroom.arm.com/news/the-arm-ecosystem-ships-a-record-6-7-billion-arm-based-chips-in-a-single-quarter.

[3] ChibiOS free embedded RTOS - ChibiOS Homepage. https://www.chibios.org/dokuwiki/doku.php.

[4] Crazyflie 2.0 | bitcraze. https://www.bitcraze.io/products/old-products/crazyflie-2-0/.

[5] embOS | Embedded RTOS | Preemptive scheduling | SEGGER | Carnica Technology. https://www.carnica-technology.com/segger-embos-rtos/embos/#1.

[6] FreeRTOS scheduling (single-core, AMP and SMP) - FreeRTOS™. https://freertos.org/Documentation/02-Kernel/02-Kernel-features/01-Tasks-and-co-routines/04-Task-scheduling.

[7] FreeRTOS™ - FreeRTOS™. https://freertos.org.

[8] The LLVM Compiler Infrastructure Project. https://llvm.org/.

[9] Loading Firmware onto Boards without Existing ArduPilot Firmware — Copter Documentation. https://ardupilot.org/copter/docs/common-loading-firmware-onto-chibios-only-boards.html.

[10] Mbed-TLS/Mbedtls. https://github.com/Mbed-TLS/mbedtls.

[11] μC/OS-III User Manual. https://www.analog.com/media/en/dsp-documentation/software-manuals/Micrium-uCOS-III-UsersManual.pdf.

[12] RISC-V Assembler Reference. https://michaeljclark.github.io/asm.html.

[13] Scheduling — Zephyr Project Documentation. https://docs.zephyrproject.org/latest/kernel/services/scheduling/index.html.

[14] tass-belgium/picotcp. https://github.com/tass-belgium/picotcp.

[15] Zephyr Project – A proven RTOS ecosystem, by developers, for developers. https://zephyrproject.org/.

[16] AMNESIA:33 Research Report Executive Summary, 2020. https://www.forescout.com/resources/amnesia33-research-report-executive-summary/.

[17] Ripple20, 2020. https://www.jsof-tech.com/disclosures/ripple20/.

[18] Mahmoud Ammar and Bruno Crispo. Verify&revive: Secure detection and recovery of compromised low-end embedded devices. In *Proceedings of the 36th Annual Computer Security Applications Conference*, pages 717–732, 2020.

[19] Arm Cortex-M Processor Comparison Table. https://developer.arm.com/documentation/102787/latest/.

[20] Arm Cortex-R Processor Comparison Table. https://developer.arm.com/documentation/102788/latest/.

[21] Jeff Arnold and M Frans Kaashoek. Ksplice: Automatic rebootless kernel updates. In *Proceedings of the 4th ACM European conference on Computer systems*, pages 187–198, 2009.

[22] N Asokan, Thomas Nyman, Norrathep Rattanavipanon, Ahmad-Reza Sadeghi, and Gene Tsudik. Assured: Architecture for secure software update of realistic embedded devices. *IEEE Transactions on Computer-Aided*

Design of Integrated Circuits and Systems, 37(11):2290–2300, 2018.

[23] Eddie Benowitz and Mark Maimone. Patching flight software on mars. In *Workshop on Spacecraft Flight Software*, volume 27, 2015.

[24] Ron Birkett. Enhancing Real-time Capabilities with the PRU. In *Embedded Linux Conference*, pages 1–39, 2015.

[25] Ferdinand Brasser, Brahim El Mahjoub, Ahmad-Reza Sadeghi, Christian Wachsmann, and Patrick Koeberl. Tytan: Tiny trust anchor for tiny devices. In *Proceedings of the 52nd annual design automation conference*, pages 1–6, 2015.

[26] Doug Brubacher. Detours: Binary interception of win32 functions. In *Windows NT 3rd symposium (windows NT 3rd symposium)*, 1999.

[27] Bryan Buck and Jeffrey K Hollingsworth. An api for runtime code patching. *The International Journal of High Performance Computing Applications*, 14(4):317–329, 2000.

[28] Haibo Chen, Rong Chen, Fengzhe Zhang, Binyu Zang, and Pen-Chung Yew. Live updating operating systems using virtualization. In *Proceedings of the 2nd international conference on Virtual execution environments*, pages 35–44, 2006.

[29] Yaohui Chen, Yuping Li, Long Lu, Yueh-Hsun Lin, Hayawardh Vijayakumar, Zhi Wang, and Xinming Ou. Instaguard: Instantly deployable hot-patches for vulnerable system programs on android. In *2018 Network and Distributed System Security Symposium (NDSS'18)*, 2018.

[30] Yue Chen, Yulong Zhang, Zhi Wang, Liangzhao Xia, Chenfu Bao, and Tao Wei. Adaptive android kernel live patching. In *26th USENIX Security Symposium (USENIX Security 17)*, pages 1253–1270, 2017.

[31] Byung-Chul Choi, Seoung-Hyeon Lee, Jung-Chan Na, and Jong-Hyouk Lee. Secure firmware validation and update for consumer devices in home networking. *IEEE Transactions on Consumer Electronics*, 62(1):39–44, 2016.

[32] Abraham A Clements, Naif Saleh Almakhdhub, Saurabh Bagchi, and Mathias Payer. {ACES}: Automatic compartments for embedded systems. In *27th USENIX Security Symposium (USENIX Security 18)*, pages 65–82, 2018.

[33] Abraham A Clements, Naif Saleh Almakhdhub, Khaled S Saab, Prashast Srivastava, Jinkyu Koo, Saurabh Bagchi, and Mathias Payer. Protecting bare-metal embedded systems with privilege overlays. In *2017 IEEE Symposium on Security and Privacy (SP)*, pages 289–303. IEEE, 2017.

[34] Scarlet Evans. IoT Malware Attacks Jump 400% Since 2022, Report. https://www.iotworldtoday.com/security/iot-malware-attacks-jump-400-since-2022-report.

[35] Solon Falas, Charalambos Konstantinou, and Maria K Michael. A modular end-to-end framework for secure firmware updates on embedded systems. *ACM Journal on Emerging Technologies in Computing Systems (JETC)*, 18(1):1–19, 2021.

[36] Sebastian Fischmeister and Klemens Winkler. Non-blocking deterministic replacement of functionality, timing, and data-flow for hard real-time systems at runtime. In *17th Euromicro Conference on Real-Time Systems (ECRTS'05)*, pages 106–114. IEEE, 2005.

[37] Wojciech Giernacki, Mateusz Skwierczyński, Wojciech Witwicki, Paweł Wroński, and Piotr Kozierski. Crazyflie 2.0 quadrotor as a platform for research and education in robotics and control engineering. In *2017 22nd International Conference on Methods and Models in Automation and Robotics (MMAR)*, pages 37–42. IEEE, 2017.

[38] Eric Gustafson, Paul Grosen, Nilo Redini, Saagar Jha, Andrea Continella, Ruoyu Wang, Kevin Fu, Sara Rampazzi, Christopher Kruegel, and Giovanni Vigna. Shimware: Toward practical security retrofitting for monolithic firmware images. In *Proceedings of the 26th International Symposium on Research in Attacks, Intrusions and Defenses*, pages 32–45, 2023.

[39] Taylor Hardin, Ryan Scott, Patrick Proctor, Josiah Hester, Jacob Sorber, and David Kotz. Application memory isolation on {ultra-Low-power}{MCUs}. In *2018 USENIX Annual Technical Conference (USENIX ATC 18)*, pages 127–132, 2018.

[40] Yi He, Zhenhua Zou, Kun Sun, Zhuotao Liu, Ke Xu, Qian Wang, Chao Shen, Zhi Wang, and Qi Li. {RapidPatch}: firmware hotpatching for {Real-Time} embedded devices. In *31st USENIX Security Symposium (USENIX Security 22)*, pages 2225–2242, 2022.

[41] Haegeon Jeong, Jeanseong Baik, and Kyungtae Kang. Functional level hot-patching platform for executable and linkable format binaries. In *2017 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 489–494. IEEE, 2017.

[42] Ralf Jung, Robbert Krebbers, Jacques-Henri Jourdan, Aleš Bizjak, Lars Birkedal, and Derek Dreyer. Iris from the ground up: A modular foundation for higher-order concurrent separation logic. *Journal of Functional Programming*, 28:e20, 2018.

[43] Trishank Karthik, Akan Brown, Sebastien Awwad, Damon McCoy, Russ Bielawski, Cameron Mott, Sam Lauzon, André Weimerskirch, and Justin Cappos. Uptane: Securing software updates for automobiles. In *International Conference on Embedded Security in Car*, pages 1–11, 2016.

[44] Chung Hwan Kim, Taegyu Kim, Hongjun Choi, Zhongshu Gu, Byoungyoung Lee, Xiangyu Zhang, and Dongyan Xu. Securing Real-Time Microcontroller Systems through Customized Memory View Switching. In *NDSS*, 2018.

[45] Haeyoung Kim, Jinjae Lee, Derry Pratama, Asep Muhamad Awaludin, Howon Kim, and Donghyun Kwon. Rimi: instruction-level memory isolation for embedded systems on risc-v. In *Proceedings of the 39th International Conference on Computer-Aided Design*, pages 1–9, 2020.

[46] Taegyu Kim, Aolin Ding, Sriharsha Etigowni, Pengfei Sun, Jizhou Chen, Luis Garcia, Saman Zonouz, Dongyan Xu, and Dave Tian. Reverse engineering and retrofitting robotic aerial vehicle control firmware using dispatch. In *Proceedings of the 20th Annual International Conference on Mobile Systems, Applications and Services*, pages 69–83, 2022.

[47] Taegyu Kim, Chung Hwan Kim, Hongjun Choi, Yonghwi Kwon, Brendan Saltaformaggio, Xiangyu Zhang, and Dongyan Xu. Revarm: A platform-agnostic arm binary rewriter for security applications. In *Proceedings of the 33rd Annual Computer Security Applications Conference*, pages 412–424, 2017.

[48] Patrick Koeberl, Steffen Schulz, Ahmad-Reza Sadeghi, and Vijay Varadharajan. Trustlite: A security architecture for tiny embedded devices. In *Proceedings of the Ninth European Conference on Computer Systems*, pages 1–14, 2014.

[49] Antonio Langiu, Carlo Alberto Boano, Markus Schuß, and Kay Römer. Upkit: An open-source, portable, and lightweight update framework for constrained iot devices. In *2019 IEEE 39th international conference on distributed computing systems (ICDCS)*, pages 2101–2112. IEEE, 2019.

[50] Euijong Lee, Young-Duk Seo, Se-Ra Oh, and Young-Gab Kim. A survey on standards for interoperability and security in the internet of things. *IEEE Communications Surveys & Tutorials*, 23(2):1020–1047, 2021.

[51] Kelun Lei, Xin You, Hailong Yang, Zhongzhi Luan, and Depei Qian. Birfia: Selective binary rewriting for function interception on arm. In *Proceedings of the 37th International Conference on Supercomputing*, pages 87–98, 2023.

[52] Njabulo S Mtetwa, Paul Tarwireyi, Adnan M Abu-Mahfouz, and Matthew O Adigun. Secure firmware updates in the internet of things: A survey. In *2019 International Multidisciplinary Information Technology and Engineering Conference (IMITEC)*, pages 1–7. IEEE, 2019.

[53] Christian Niesler, Sebastian Surminski, and Lucas Davi. HERA: Hotpatching of Embedded Real-time Applications. In *NDSS*, 2021.

[54] Dennis K Nilsson and Ulf E Larson. Secure firmware updates over the air in intelligent vehicles. In *ICC Workshops-2008 IEEE International Conference on Communications Workshops*, pages 380–384. IEEE, 2008.

[55] NIST. NVD - CVE-2017-2784. https://nvd.nist.gov/vuln/detail/CVE-2017-2784.

[56] NIST. NVD - CVE-2020-10024. https://nvd.nist.gov/vuln/detail/CVE-2020-10024.

[57] Daniele Perito and Gene Tsudik. Secure code update for embedded devices via proofs of secure erasure. In *Computer Security–ESORICS 2010: 15th European Symposium on Research in Computer Security, Athens, Greece, September 20-22, 2010. Proceedings 15*, pages 643–662. Springer, 2010.

[58] Ani Petrosyan. Monthly Number of IoT Attacks Global 2022. https://www.statista.com/statistics/1322216/worldwide-internet-of-things-attacks/.

[59] Prashant Hari Narayan Rajput, Constantine Doumanidis, and Michail Maniatakos. {ICSPatch}: Automated Vulnerability Localization and {Non-Intrusive} Hotpatching in Industrial Control Systems using Data Dependence Graphs. In *32nd USENIX Security Symposium (USENIX Security 23)*, pages 6861–6876, 2023.

[60] Ashwin Ramaswamy, Sergey Bratus, Sean W Smith, and Michael E Locasto. Katana: A hot patching framework for elf executables. In *2010 International Conference on Availability, Reliability and Security*, pages 507–512. IEEE, 2010.

[61] Ted Romer, Geoff Voelker, Dennis Lee, Alec Wolman, Wayne Wong, Hank Levy, Brian Bershad, and Brad Chen. Instrumentation and optimization of win32/intel executables using etch. In *Proceedings of the USENIX Windows NT Workshop*, volume 1997, pages 1–8, 1997.

[62] Mohsen Salehi and Karthik Pattabiraman. AutoPatch: Automated Generation of Hotpatches for Real-Time Embedded Devices. *arXiv preprint arXiv:2408.15372*, 2024.

[63] Christopher Salls, Yan Shoshitaishvili, Nick Stephens, Christopher Kruegel, and Giovanni Vigna. Piston: Uncooperative remote runtime patching. In *Proceedings of the 33rd Annual Computer Security Applications Conference*, pages 141–153, 2017.

[64] Philipp Scholl, Felix Sutton, and Philipp Sommer. Secure On-The-Fly Firmware Updates for Safety Critical Embedded Systems. In *2024 IEEE 29th International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 1–7. IEEE, 2024.

[65] Simon Segars. Arm Partners Have Shipped 200 Billion Chips. https://newsroom.arm.com/blog/200bn-arm-chips.

[66] Ben Seri, Gregory Vishnepolsky, and Dor Zusman. Critical vulnerabilities to remotely compromise VxWorks, the most popular RTOS. *White Paper, ARMIS, URGENT/11*, 2019.

[67] Arvind Seshadri, Mark Luk, Adrian Perrig, Leendert Van Doorn, and Pradeep Khosla. SCUBA: Secure code update by attestation in sensor networks. In *Proceedings of the 5th ACM workshop on Wireless security*, pages 85–94, 2006.

[68] Dmitry Sklyar. ChargePoint Home Security Research, 2018. https://media.kasperskycontenthub.com/wp-content/uploads/sites/43/2018/12/13084354/ChargePoint-Home-security-research_final.pdf.

[69] Lionel Sujay Vailshery. IoT connections worldwide 2022-2033, 2024. https://www.statista.com/statistics/1183457/iot-connected-devices-worldwide/.

[70] Michael Wahler, Stefan Richter, Sumit Kumar, and Manuel Oriol. Non-disruptive large-scale component updates for real-time controllers. In *2011 IEEE 27th International Conference on Data Engineering Workshops*, pages 174–178. IEEE, 2011.

[71] Jean-Luc Watson, Saharsh Agrawal, Ryan Tsang, Sherry Luo, Raluca Ada Popa, and Prabal Dutta. Retcon: Live Updates for Embedded Event-Driven Applications. In *2024 23rd ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*, pages 126–137. IEEE, 2024.

[72] Yuhao Wu, Jinwen Wang, Yujie Wang, Shixuan Zhai, Zihan Li, Yi He, Kun Sun, Qi Li, and Ning Zhang. Your firmware has arrived: A study of firmware update vulnerabilities. In *33rd USENIX Security Symposium (USENIX Security 24)*, pages 5627–5644, 2024.

[73] Zhengzi Xu, Yulong Zhang, Longri Zheng, Liangzhao Xia, Chenfu Bao, Zhi Wang, and Yang Liu. Automatic hot patch generation for android kernels. In *29th USENIX Security Symposium (USENIX Security 20)*, pages 2397–2414, 2020.

[74] Zhenyu Ye, Lei Zhou, Fengwei Zhang, Wenqiang Jin, Zhenyu Ning, Yupeng Hu, and Zheng Qin. FortifyPatch: Towards Tamper-Resistant Live Patching in Linux-Based Hypervisor. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 38–50, 2024.

[75] Koen Zandberg, Kaspar Schleiser, Francisco Acosta, Hannes Tschofenig, and Emmanuel Baccelli. Secure firmware updates for constrained iot devices using open standards: A reality check. *IEEE access*, 7:71907–71920, 2019.

[76] Ming Zhou, Haining Wang, Ke Li, Hongsong Zhu, and Limin Sun. Save the Bruised Striver: A Reliable Live Patching Framework for Protecting Real-World PLCs. In *Proceedings of the Nineteenth European Conference on Computer Systems*, pages 1192–1207, 2024.

[77] Xia Zhou, Jiaqi Li, Wenlong Zhang, Yajin Zhou, Wenbo Shen, and Kui Ren. OPEC: operation-based security isolation for bare-metal embedded systems. In *Proceedings of the Seventeenth European Conference on Computer Systems*, pages 317–333, 2022.

A Memory Enforcement Algorithm

The algorithm uses two boolean inputs, M_P (indicating if the last task was the *Manager*) and M_N (indicating if the next task is the *Manager*). Both cannot be true simultaneously since context switches may only occur when tasks differ. If M_N is true (line 1), the *Guard* grants write access to the applicator context (line 2), shared by both the *Manager* and *Applicator*. If a hotpatch is scheduled (line 3), the *Guard* grants write access to the firmware, invokes the *Applicator*, and revokes access post-execution (lines 4-6). The *Guard* also prepares for the *Manager* by granting write access to hotpatch slots (lines 8-9). Conversely, if M_P is true (line 10), indicating the previous task was the *Manager*, the *Guard* revokes all memory access rights, restoring a default read-only state (lines 11-13) to prevent lingering permissions post-execution.

B Hotpatch Source Code Example

In this example, we discuss the vulnerability `CVE-2017-2784`, which involves improper handling of an invalid free stack pointer, potentially leading to remote code execution during x509 ECDSA parsing. To mitigate this, it's advised to avoid the *secp224k1* curve. The hotpatch prevents its use by returning an error whenever the firmware attempts to access it. The hotpatch code, shown in Listing 1, modifies the `mbedtls_ecp_check_pubkey` function. It checks if the group's identifier is `MBEDTLS_ECP_DP_SECP224K1` and, if so, returns `MBEDTLS_ERR_ECP_FEATURE_UNAVAILABLE`. If the identifier doesn't match, it executes the overwritten original code and returns to the function.

Listing 1: Example Hotpatch code for `CVE-2017-2784`.

```
#include "mbedtls/ecp.h"
#include "hp_helper.h"
#pragma GCC push_options
#pragma GCC optimize ("Os")
HOTPATCH_FUNCTION_START_NAKED(
    mbedtls_ecp_check_pubkey, 4, redirect, 8) {
    STORE_REGISTERS("r3");
    VAR_REG(grp, "r0", const mbedtls_ecp_group *);

    if (grp->id == MBEDTLS_ECP_DP_SECP224K1) {
        SET_RETURN_VALUE(
            MBEDTLS_ERR_ECP_FEATURE_UNAVAILABLE);
        RESTORE_REGISTERS("r3-r8, pc");
    }
    RESTORE_REGISTERS("r3");
        ORIGINAL_CODE();
    HOTPATCH_BRANCH_TO_ORIGINAL_CODE();
}
#pragma GCC pop_options
```

C Security Analysis and Modeling

We complement the formal analysis in Section 7 with some additional details of interest.

Our formal model of KINTSUGI mirrors the pseudocode versions of the C code, e.g., Algorithm 1 for the *Guard*. The structure of the remaining components is given in Algorithm 2 for the *Manager* and Algorithm 3 for the *Applicator*.

As we remark elsewhere, the formal analysis is not directly concerned with the real-time behavior of the RTOS. Rather, the security properties concern themselves with the *integrity* of the hotpatching process and arise as a consequence of the proper management of shared memory resources, which can naturally be modeled using separation logic. To reason formally about performance and schedulability we would need to equip our semantics with a notion of time and more details about the timing characteristics of the hardware.

Post-Application Hotpatch Integrity. The most interesting parts about the *security proofs* are outlined in Section 7. The *correctness* of the framework's code is usually straightforward. The most important use of the correctness of

Algorithm 2 Pseudocode of the *Manager* functionality.

Input: $input_buffer$
Resources: ρ_{hs}
Private data: $quarantine_zone$, $valid$, $verified$

1: $receive_hotpatch(input_buffer)$;
2: $copy_hotpatch(input_buffer, quarantine_zone)$;
3: $valid = validate_patch(quarantine_zone)$;
4: $verified = verified_patch(quarantine_zone)$;
5: **if** $valid$ && $verified$ **then**
6: $\quad schedule_hotpatch(quarantine_zone, \rho_{hs})$;
7: $\quad set_hotpatch_scheduled()$;
8: **end if**
9: $erase(quarantine_zone)$;
10: **while** $hotpatch_scheduled()$ **do**
11: **end while**

$schedule_hotpatch()$
Private data: i

1: **if** $i = next_available_slot(\rho_{hs})$ **then**
2: $\quad slot_add_hotpatch(i, \rho_{hs}, quarantine_zone)$;
3: $\quad set_slot_pending(i)$;
4: $\quad$ **if** $hotpatch_scheduled()$ **then**
5: $\quad\quad fail()$;
6: $\quad$ **else**
7: $\quad\quad finalize_hotpatch()$;
8: $\quad\quad set_context_active()$;
9: $\quad$ **end if**
10: **else**
11: $\quad fail()$;
12: **end if**

Algorithm 3 Pseudocode of the *Applicator* functionality.

Resources: ρ_{hs}
Private data: i

1: $i = get_slot_pending()$;
2: $transfer_hotpatch_to_firmware(i)$;
3: $set_slot_active(i)$;
4: $unset_hotpatch_scheduled()$;

KINTSUGI's code is the establishment of hotpatch slot lifetime rules for Goal 4. Here we show the essence of the correctness argument for the case without rollbacks.

The dedicated region that holds all the state of the framework, ρ_{hs}, includes a given number of hotpatch slots. Alongside each of these slots, say i, there is a flag to keep track of its current state. The framework uses this information to determine what is the next available slot when applying a new patch. The correctness of these operations is easy to establish if we can rule out interference from the attacker (as we can conclude from previous security goals). All this work takes place exclusively within the protected region ρ_{hs}.

Initially, all hotpatch slots are *inactive*, and we need to establish how they become active, and what happens afterwards throughout a whole-system execution $s_0 \rightarrow_{seq} s_f$ that interleaves kernel and user code (including the *Manager*'s and attacker-controlled tasks). As a consequence of Goal 2 and Goal 3, we know that the attacker cannot ever influence the contents of ρ_{hz} (and in our strong model this rules out interference from *all* user tasks). So all we need to show is the lifetime rules for hotpatches just from the framework's code.

During initialization, the state of all hotpatches is set to inactive, and no hotpatches are scheduled. The *Applicator* is only called by the *Guard* when a hotpatch is scheduled. The only component that can trigger this is the *Manager*, which sets the flag in line 7 after scheduling a hotpatch. This involves using $next_available_slot()$ to select an inactive slot i, transferring the hotpatch date from the quarantine zone and then setting its state to *pending*. After this (there should be no hotpaches scheduled yet), $finalize_hotpatch()$ completes preparations, sets the context flag to run the *Applicator*, and finally the hotpatch slot is set to *scheduled*. At this point, the *Manager* task is on hold until a new hotpatch can be scheduled, but because of this flag the *Guard* can finally call the *Applicator*, which effects the changes, sets the hotpatch slot to *active* and clears its control flag.

Because the progression of hotpatch states is irreversible until reboot (barring rollbacks), and the helpers will never select an active hotpatch slot after it is in use, we can conclude their integrity after application.

D Complete Micro-Benchmarking Table

We provide the complete measurements for each hotpatch size from `RapidPatch` [40] and `AutoPatch` [62] in Table 5.

E Real-World CVE Micro-Benchmarks

We also conducted micro-benchmarking experiments on the hotpatch sizes from our developed hotpatches, as shown in Table 6, in addition to the benchmarks provided by `RapidPatch` [40] and `AutoPatch` [62].

Table 5: Complete measurement results of the micro-benchmarks shown in Section 6.1 considering all hotpatch sizes demonstrated in `RapidPatch` and `AutoPatch`.

	Hotpatch Code Size (Bytes)	Manager		Validation		Storing		Scheduling	
		Cycles	Time (μs)	Cycles	Time (μs)	Cycles	Time (μs)	Cycles	Time (μs)
RapidPatch	48	2507	39.17	47	0.73	874	13.66	118	1.84
	52	2645	41.33	47	0.73	928	14.50	118	1.84
	56	2755	43.05	47	0.73	954	14.91	118	1.84
	68	3141	49.08	47	0.73	1088	17.00	118	1.84
	124	4877	76.20	47	0.73	1648	25.75	118	1.84
	156	5869	91.70	47	0.73	1968	30.75	118	1.84
	188	6861	107.20	47	0.73	2288	35.75	118	1.84
	232	8211	128.30	47	0.73	2714	42.41	118	1.84
	260	9092	142.06	47	0.73	3006	46.97	117	1.83
AutoPatch	380	12812	200.19	47	0.73	4206	65.72	117	1.83
	392	13169	205.77	47	0.73	4311	67.36	117	1.83
	400	13417	209.64	47	0.73	4391	68.61	117	1.83
	476	15788	246.69	47	0.73	5166	80.72	117	1.83
	504	16641	260.02	47	0.73	5431	84.86	117	1.83
	516	17028	266.06	47	0.73	5566	86.97	117	1.83
	528	17385	271.64	47	0.73	5671	88.61	117	1.83
	548	18020	281.56	47	0.73	5886	91.97	117	1.83
	560	18377	287.14	47	0.73	5991	93.61	117	1.83
	780	25212	393.94	47	0.73	8206	128.22	117	1.83
	944	30281	473.14	47	0.73	9831	153.61	117	1.83

Table 6: CVEs implemented and mitigated by KINTSUGI together with their measured micro-benchmarks.

CVE-ID	Manager		Validation		Storing		Scheduling	
	Cycles	Time (μs)	Cycles	Time (μs)	Cycles	Time (μs)	Cycles	Time (μs)
CVE-2020-10021	4243	66.30	47	0.73	1434	22.41	118	1.84
CVE-2020-10023	2259	35.30	47	0.73	794	12.41	118	1.84
CVE-2020-10024	1763	27.55	47	0.73	634	9.91	118	1.84
CVE-2020-10062	5483	85.67	47	0.73	1834	28.66	118	1.84
CVE-2020-10063	4801	75.02	47	0.73	1614	25.22	118	1.84
CVE-2018-16524	4133	64.58	47	0.73	1408	22.00	118	1.84
CVE-2018-16603	2893	45.20	47	0.73	1008	15.75	118	1.84
CVE-2017-2784	3003	46.92	47	0.73	1034	16.16	118	1.84
CVE-2020-17443	2755	43.05	47	0.73	954	14.91	118	1.84
CVE-2020-1744	3389	52.95	47	0.73	1168	18.25	118	1.84

F Qualitative Comparison

We provide a qualitative comparison of KINTSUGI against state-of-the-art hotpatching frameworks across characteristics as well as security properties in Table 7.

Table 7: Qualitative comparison between KINTSUGI and state-of-the-art hotpatching frameworks.
I = Instruction-Level, *F* = Function-Level.
○= Not Considered, ◐= Partially Considered, ●= Considered

	FW Exec.	HP Exec.	HP Gran.	HW Ind.	V&V	TR	PAI
`HERA` [53]	Flash	Native	*I*	Yes	○	◐	○
`RapidPatch` [40]	Flash	eBPF	*F*	No	○	○	○
`AutoPatch` [62]	Flash	Native	*F*	No	○	○	○
`Kintsugi`	RAM	Native	*I*	Yes	●	●	●

FW Exec. = Firmware Execution, **HP Exec.** = Hotpatch Execution, **HP Gran.** = Hotpatch Granularity, **HW Ind.** = Hardware Independent, **V&V**: Verification & Validation, **TR**: Tamper Resistance, **PAI**: Post-Application Integrity.